

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2ème_année_AN_01.pdf

Instructor : BELACEL Madani

Institution : École Normale Supérieure de Mostaganem

Module : Computer Science

Level : ENS — Year 2

Duration : 1h30

Academic Year : 2025-2026

Course sheet — E-learning platform (Moodle)

Reference : benchmark 10 Algerian universities + USTO/Biskra/M'sila practices.

Target audience : ENS 1st and 2nd year students — French and English Departments

Prerequisite skills : Baccalaureate ; ICT Bachelor Level 1 recommended

Cross-cutting objectives :

- Understand machine architecture and operating systems
- Design algorithms and elementary data structures
- Master networking principles and basic computer security
- Get started with Python for teaching

Assessment method : Tutorial 40 % + Python Practical 35 % + MCQ 25 %

Bibliography : Computer Architecture 1 — Univ. Biskra ; Computer Security S6 — Univ. M'sila ; Goupille — Computer Technology

1) Learning Objectives

After this module, the ENS 2nd year student is able to :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python.
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity.
- Model data with the relational model and basic SQL queries.
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP).
- Use Git and collaborative work methods (branches, merge).
- Prepare for teaching computer science in middle/high school (Scratch, pedagogical Python).
- Articulate computer science, internship and digital teaching portfolio.

2) Course Plan

1. ENS1 Review and skill building

2. Object-oriented programming
3. Advanced algorithms and recursion
4. Databases and SQL
5. Fundamental Web development
6. Git and software engineering intro
7. Didactics of computer science at school
8. Integrated ENS2 project

3) Detailed content

```

timeline
title Computer Science Module Path – ENS Year 2
Session 01 : Module presentation : Objectives and plan
Sessions 02-03 : Python OOP : Classes, inheritance, polymorphism
Session 04 : Advanced Algorithms : Recursion, sorts, complexity
Session 05 : Databases : Relational model, SQL
Session 06 : Web Development : HTML, CSS, JavaScript
Session 07 : Git : Versions, branches, collaboration
Session 08 : Didactics : Teaching computer science
Sessions 09-10 : Integrated project : Full application + defense

```

```

mindmap
root((Computer Science Module ENS2))
  Programming
  Python OOP
  Classes & Objects
  Inheritance
  Algorithms
  Recursion
  Sorts (bubble, merge)
  Binary search
  Data
  Relational model
  SQLite
  SQL queries
  Web
  HTML5
  CSS3
  JavaScript
  Tools
  Git & GitHub
  Flask
  Debugging
  Pedagogy
  Didactics
  Scratch
  Lesson plan

```

Chapter 1 — ENS2 Review and Challenges (10 min)

Transition from isolated script to structured software project. Future teacher skills : algorithms, programming, digital culture, AI ethics.

Chapter 2 — OOP (20 min)

Class, object, attribute, method. Constructor `__init__`. Encapsulation, simple inheritance. Example : `class Eleve`, `class Enseignant(Eleve)` — adapted to the school context.

Chapter 3 — Advanced Algorithms (15 min)

Bubble sort, merge sort (notion), binary search (sorted array). Recursion : factorial, Fibonacci. Complexity comparison.

Chapter 4 — Databases (15 min)

Relational model : table, primary key, foreign key. SQL : SELECT, WHERE, JOIN, INSERT. Integrity and normalisation (1NF intro).

Chapter 5 — Web (15 min)

HTML structure, CSS styling, simple JavaScript event. Client-server, forms, REST API (notion). Web accessibility.

Chapter 6 — Git and Quality (10 min)

git init, add, commit, branch, merge. README, .gitignore. Unit tests intro (assert).

Chapter 7 — Computer Science Didactics (10 min)

Official curricula, project approach, Scratch → Python. Differentiation, competency-based assessment. AI usage charter in class.

Chapter 8 — ENS2 Project (5 min)

Deliverable : lightweight Web application or Python OOP module + SQLite database + Git repository + 45 min pedagogical lesson plan for middle school.

4) Assessment Methods

Component	Weight
OOP + SQL Practical	35 %
Integrated project (code + Git)	40 %
Pedagogical lesson plan	25 %

5) Bibliography

- Guttag — Introduction to Computation and Programming Using Python.
- Connolly & Begg — Database Systems (introductory chapters).
- MDN Web Docs — HTML, CSS, JavaScript.
- Eduscol — Teaching computer science in high school.

MCQ Self-Assessment

1. What is the weight of the integrated project in the final assessment?
a) 25 % b) 40 % ✓ c) 35 %
2. Which session introduces object-oriented programming?
a) Session 01 b) Session 02 ✓ c) Session 04
3. The NSI module in high school has a weekly volume of :

a) 1h30/week b) 4h/week ✓ c) 2h/week

4. Which versioning tool is taught in this module ?

a) SVN b) Git ✓ c) Mercurial

5. Which language is used for the Scratch → text pedagogical transition ?

a) Java b) Python ✓ c) C++

Conclusion

This module accompanies you from technical mastery (OOP, algorithms, databases, Web, Git) to didactic transposition for secondary education. Each session builds a brick of the integrated project that will consolidate all targeted skills.

Next lesson : Object-oriented programming — classes, objects and methods in Python.

Pedagogical Note ENS

Teaching transposition : The module overview can be presented to middle/high school students in the form of a mind map. From 9th grade onward, one can illustrate the link between Scratch blocks (visual programming) and Python code (textual programming) to show the progression of learning. In 10th grade SNT, the "Internet" theme can be linked to the Web session ; the "Structured Data" theme to the SQL session.

Trainer advice : Use this course plan as a communication tool with your students. Distribute it at the beginning of the module so they can visualize the overall coherence. Have them check off sessions as they go, which reinforces the sense of progress. Do not hesitate to revisit this plan at the start of each session to contextualize.

Extension : Students can create a digital mind map (XMind, FreeMind) of their own progress in the module, adding personal resources (links, notes, exercises) — this will become a foundation for their teaching portfolio.

Templates : [Template/PG-fr.docx](#) | [Template/Presentation_BELACEL_V3.pptx](#)

PDF : [Cours_Informatique_ENS_2ème_année_AN_02.pdf](#)

Computer Science Course — ENS Year 2 — Session 02 (EN)

Instructor : BELACEL Madani

Module : Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic Year : 2025-2026

Course sheet — E-learning platform (Moodle)

Reference : benchmark 10 Algerian universities + USTO/Biskra/M'sila practices.

Target audience : ENS 2nd year students — English Department

Prerequisite skills : Python basics (ENS1 session 09), algorithms (ENS1 session 05)

Cross-cutting objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare for teaching computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP + SQL Practical 35 % — Integrated project 40 % — Pedagogical lesson plan 25 %

Bibliography : Guttag — Introduction to Computation and Programming Using Python ; Lutz — Learning Python

Specific Learning Objectives

After this session, the student is able to :

- Define a Python class with attributes and methods
- Instantiate objects
- Use the `__init__` constructor
- Understand the difference between class and instance
- Create a simple class adapted to a school context

Course Plan

1. Procedural vs object-oriented paradigm
2. Class and object : fundamental concepts
3. Attributes and methods

4. `__init__` constructor
5. Class vs instance variables
6. Example : Eleve class
7. Example : Evaluation class

1. Why OOP ? (10 min)

Procedural programming (ENS1) : functions + separate data.

Object-oriented programming : groups data and behaviour into objects.

Advantages :

- Natural organisation (we think in terms of real-world objects)
- Reusable (a class = a mould for objects)
- Maintainable (modifying a class does not affect the rest)
- Extensible (inheritance)

Concrete example : in school life, we handle Students, Teachers, Grades, Classes — these are objects.

```

flowchart LR
  A[Real problem<br/>school management] --> B[Identify objects<br/>Student, Grade, Class]
  B --> C[Define classes<br/>attributes + methods]
  C --> D[Instantiate objects]
  D --> E{Consistent object ?}
  E -->|Yes| F[Reuse<br/>in other contexts]
  E -->|No| B
  F --> G[Maintenance<br/>and evolution]
```

2. Class and object (15 min)

Class : model, construction plan.

Object : concrete realisation of the model (instance).

```

# Class definition
class Eleve:
    pass

# Creating objects (instances)
elevel = Eleve()
eleve2 = Eleve()

print(type(elevel)) # <class '__main__.Eleve'>
```

The class is the mould ; the object is the element produced by this mould.

```

classDiagram
class Eleve {
+string ecole
+int total_eleves
+string nom
+int age
+__init__(nom, age)
+se_presenter()
}
class EleveAvance {
```

```

+list notes
+string groupe
+ajouter_note(note)
+moyenne()
+afficher()
}
Eleve <|-- EleveAvance
note for Eleve "Class = construction plan"
note for EleveAvance "Object = concrete instance"

```

3. Attributes and methods (15 min)

Attribute : variable attached to an object (its state).

Method : function attached to an object (its behaviour).

```

class Eleve:
    def __init__(self, nom, age):
        self.nom = nom # Instance attribute
        self.age = age

    def se_presenter(self): # Method
        print(f"Je m'appelle {self.nom} et j'ai {self.age} ans.")

# Usage
e = Eleve("Ali", 20)
print(e.nom) # Access attribute
e.se_presenter() # Call method

```

Note : self represents the current instance — it is the first parameter of every method.

4. __init__ constructor (15 min)

The __init__ method is automatically called when the object is created.

```

class Evaluation:
    def __init__(self, matiere, note, coefficient=1):
        self.matiere = matiere
        self.note = note
        self.coefficient = coefficient

    def ponderer(self):
        return self.note * self.coefficient

# Creation
eval1 = Evaluation("Maths", 15, 2)
eval2 = Evaluation("French", 12)
print(eval1.ponderer()) # 30
print(eval2.ponderer()) # 12 (default coefficient = 1)

```

5. Class variables (10 min)

Attributes shared by all instances.

```

class Eleve:
    ecole = "ENS Mostaganem" # Class variable

    def __init__(self, nom):
        self.nom = nom # Instance variable

```

```
e1 = Eleve("Ali")
e2 = Eleve("Sara")
print(e1.ecole) # ENS Mostaganem
print(e2.ecole) # ENS Mostaganem
```

Use : constants, counters, shared configuration.

6. Complete example : Eleve class (20 min)

```
class Eleve:
    ecole = "ENS Mostaganem"
    total_eleves = 0 # Shared counter

    def __init__(self, nom, prenom, groupe):
        self.nom = nom
        self.prenom = prenom
        self.groupe = groupe
        self.notes = []
        Eleve.total_eleves += 1

    def ajouter_note(self, note):
        if 0 <= note <= 20:
            self.notes.append(note)
        else:
            print("Invalid grade")

    def moyenne(self):
        if self.notes:
            return sum(self.notes) / len(self.notes)
        return 0.0

    def afficher(self):
        print(f"{self.prenom} {self.nom} – {self.groupe}")
        print(f" Grades : {self.notes}")
        print(f" Average : {self.moyenne():.2f}")

# Test
e = Eleve("Belacel", "Ali", "ENS1-A")
e.ajouter_note(15)
e.ajouter_note(12)
e.ajouter_note(18)
e.afficher()
print(f"Total students : {Eleve.total_eleves}")
```

7. Guided exercise : Matiere class (10 min)

Create a Matiere class with :

- Attributes : nom, coefficient, nb_heures
- `__init__` method with parameters
- `afficher()` method that displays the information
- `est_importante()` method that returns True if coefficient > 2

```
class Matiere:
    def __init__(self, nom, coefficient, nb_heures):
        self.nom = nom
        self.coefficient = coefficient
        self.nb_heures = nb_heures
```



```
def afficher(self):  
    print(f"{self.nom} (coef {self.coefficient}, {self.nb_heures}h)")  
  
def est_importante(self):  
    return self.coefficient > 2
```

MCQ Self-Assessment

1. What is a class in OOP ?
a) A global variable b) A construction plan for creating objects ✓ c) A special function
2. Which keyword represents the current instance in a Python method ?
a) this b) self ✓ c) instance
3. The `__init__` method is :
a) Called manually by the programmer b) Called automatically when the object is created ✓
c) Optional for every class
4. What is the difference between a class variable and an instance variable ?
a) The class variable is shared by all instances ✓ b) The instance variable is shared c) There is no difference
5. What does `Eleve.total_eleves` return in the course example ?
a) The number of methods of the class b) The total number of instances created ✓ c) The school name

8. Conclusion (5 min)

OOP organises code around objects that encapsulate data and behaviour. Classes are reusable and easy to maintain.

Next lesson : Inheritance and polymorphism in Python.

Pedagogical Note ENS

Teaching transposition : The notion of an object can be introduced in middle school without a computer through an unplugged activity : each student receives a "living being" card with fields to fill (name, age, colour). The blank card is the class, the filled card is the object. In 11th grade NSI, one can show that the `Eleve` class from the course is similar to a student record form they fill each year. The parallel with a spreadsheet (row = object, column = attribute) facilitates understanding.

Trainer advice : When grading practical work, insist on the class/instance distinction : many students confuse `Eleve` (the class) and `e` (the instance). An effective exercise is to ask "What is the type of `e` ?" and "What is the type of `Eleve` ?". Also propose an exercise where students must add a `date_naissance` attribute and an `age()` method to consolidate understanding.

Extension : Students can create a `SalleDeClasse` class that contains a list of `Eleve` objects and methods to compute the class average, list students by group, or export data to CSV format. This mini-project prepares the integrated end-of-module project.

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2ème_année_AN_03.pdf

Computer Science Course — ENS Year 2 — Session 03 (EN)

Instructor : BELACEL Madani

Module : Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic Year : 2025-2026

Course sheet — E-learning platform (Moodle)

Reference : benchmark 10 Algerian universities + USTO/Biskra/M'sila practices.

Target audience : ENS 2nd year students — English Department

Prerequisite skills : Python OOP — classes, objects, methods (session 02)

Cross-cutting objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare for teaching computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP + SQL Practical 35 % — Integrated project 40 % — Pedagogical lesson plan 25 %

Bibliography : Guttag — Introduction to Computation and Programming Using Python ; Lutz — Learning Python

Specific Learning Objectives

After this session, the student is able to :

- Make one class inherit from another class
- Override a method
- Use `super()` to call the parent class
- Protect attributes with encapsulation
- Explain polymorphism with a concrete example

Course Plan

1. Inheritance : principle
2. Parent class and child class
3. Method overriding

4. The super() function
5. Multiple inheritance (notion)
6. Encapsulation and properties
7. Polymorphism
8. Complete example : school hierarchy

1. Inheritance (15 min)

Principe : a class can inherit attributes and methods from another class.

Vocabulary :

- Parent class (super-class) : base class
- Child class (sub-class) : inherits and can add/modify

```
class Personne:
    def __init__(self, nom, prenom):
        self.nom = nom
        self.prenom = prenom

    def se_presenter(self):
        return f"{self.prenom} {self.nom}"

class Eleve(Personne): # Eleve inherits from Personne
    def __init__(self, nom, prenom, groupe):
        super().__init__(nom, prenom) # Call parent constructor
        self.groupe = groupe
```

```
e = Eleve("Benali", "Sara", "ENS2-A")
print(e.se_presenter()) # Inherited ! → "Sara Benali"
print(e.groupe) # Specific to Eleve → "ENS2-A"
```

```
classDiagram
class Personne {
+string nom
+string prenom
+__init__(nom, prenom)
+se_presenter()
+description()
}
class Eleve {
+string groupe
+list notes
+__init__(nom, prenom, groupe)
+ajouter_note(note, matiere)
+moyenne_generale()
}
class Enseignant {
+string matiere
+__init__(nom, prenom, email, matiere)
+se_presenter()
}
class IntervenantExterne {
+se_presenter()
}
Personne <|-- Eleve : inherits
Personne <|-- Enseignant : inherits
Personne <|-- IntervenantExterne : inherits
note for Personne "Super-class"
note for Eleve "Sub-class"
```

```
note for Enseignant "Overrides se_presenter()"
```

2. Override (10 min)

A child class can redefine a method from the parent class.

```
class Personne:
    def description(self):
        return "A person"

class Enseignant(Personne):
    def __init__(self, nom, matiere):
        self.nom = nom
        self.matiere = matiere

    def description(self): # Override
        return f"Teacher of {self.matiere}"

p = Personne()
e = Enseignant("Mr. Belacel", "Computer Science")
print(p.description()) # "A person"
print(e.description()) # "Teacher of Computer Science"
```

3. The super() function (10 min)

super() allows calling a method from the parent class.

```
class Enseignant(Personne):
    def __init__(self, nom, prenom, matiere):
        super().__init__(nom, prenom) # Calls Personne.__init__
        self.matiere = matiere

    def se_presenter(self):
        # Extends the parent method
        base = super().se_presenter()
        return f"{base} – Teacher of {self.matiere}"
```

4. Multiple inheritance (10 min)

Python allows inheriting from multiple classes (beware of conflicts).

```
class Sportif:
    def discipline(self):
        return "Sport"

class Musicien:
    def instrument(self):
        return "Piano"

class EleveSportifMusicien(Personne, Sportif, Musicien):
    pass

e = EleveSportifMusicien("Ali", "Ben")
print(e.se_presenter()) # From Personne
print(e.discipline()) # From Sportif
print(e.instrument()) # From Musicien
```

5. Encapsulation (15 min)

Encapsulation : protecting access to attributes to guarantee integrity.

```
class CompteBancaire:
    def __init__(self, titulaire, solde=0):
        self.titulaire = titulaire
        self._solde = 0 # Protected (convention : _)
        if solde >= 0:
            self._solde = solde

    def deposer(self, montant):
        if montant > 0:
            self._solde += montant

    def retirer(self, montant):
        if 0 < montant <= self._solde:
            self._solde -= montant
        return montant
        return 0

    @property # Getter
    def solde(self):
        return self._solde

    @solde.setter # Setter
    def solde(self, valeur):
        if valeur >= 0:
            self._solde = valeur

# Usage
cb = CompteBancaire("Ali", 1000)
cb.deposer(500)
print(cb.solde) # 1500 (via getter)
# cb._solde = 999999 # Technically possible but discouraged
```

Python convention : `_attribut` = protected, `__attribut` = private (name mangling).

```
flowchart LR
    subgraph Encapsulation
        A[Private attribute _solde] --> B[Getter @property]
        A --> C[Setter @solde.setter]
        B --> D[Controlled reading]
        C --> E[Writing with validation]
    end
    subgraph Polymorphism
        F[Function faire_cours] --> G[Enseignant.se_presenter]
        F --> H[IntervenantExterne.se_presenter]
        G --> I[Same interface<br/>different behaviours]
        H --> I
    end
```

6. Polymorphism (10 min)

Polymorphism : the same interface can have different behaviours depending on the object type.

```
def faire_cours(intervenant):
    """Polymorphic function"""
    print(intervenant.se_presenter())
    print(" will teach.")

class Enseignant(Personne):
    def se_presenter(self):
```

```

return f"{self.prenom} {self.nom} (teacher)"

class IntervenantExterne(Personne):
def se_presenter(self):
return f"{self.prenom} {self.nom} (external speaker)"

# Same function, different behaviours
faire_cours(Enseignant("Belacel", "Madani"))
faire_cours(IntervenantExterne("Dupont", "Jean"))

```

7. Complete example : school system (15 min)

```

class Personne:
def __init__(self, nom, prenom, email):
self.nom = nom
self.prenom = prenom
self.email = email

def __str__(self):
return f"{self.prenom} {self.nom}"

class Eleve(Personne):
def __init__(self, nom, prenom, email, groupe):
super().__init__(nom, prenom, email)
self.groupe = groupe
self.notes = []

def ajouter_note(self, note, matiere):
self.notes.append({"matiere": matiere, "note": note})

def moyenne_generale(self):
if not self.notes:
return 0.0
return sum(n["note"] for n in self.notes) / len(self.notes)

class Enseignant(Personne):
def __init__(self, nom, prenom, email, matiere):
super().__init__(nom, prenom, email)
self.matiere = matiere

def __str__(self):
return f"{self.prenom} {self.nom} – {self.matiere}"

# Test
eleve = Eleve("Benali", "Sara", "s.benali@etu.ens.dz", "ENS2-A")
eleve.ajouter_note(15, "CS")
eleve.ajouter_note(12, "French")
prof = Enseignant("Belacel", "Madani", "m.belacel@ens.dz", "Computer Science")
print(eleve) # "Sara Benali"
print(prof) # "Madani Belacel – Computer Science"
print(f"Average : {eleve.moyenne_generale():.2f}")

```

MCQ Self-Assessment

- What does inheritance allow in OOP ?
 - Create a class from another class ✓
 - Delete attributes
 - Prevent code modification
- Which keyword allows calling the parent class constructor ?

a) `parent()` b) `super()` ✓ c) `base()`

3. Encapsulation allows :

a) Making code faster b) Protecting access to attributes ✓ c) Removing methods

4. What does polymorphism mean ?

a) A class can have multiple parents b) The same interface can have different behaviours depending on the object ✓ c) Code runs more slowly

5. What is the Python convention for a protected attribute ?

a) `_attribut` ✓ b) `__attribut__` c) `attribut_`

Conclusion

Inheritance, encapsulation and polymorphism are the three pillars of OOP. They allow writing structured, reusable and extensible code.

Next lesson : Advanced algorithms and recursion.

Pedagogical Note ENS

Teaching transposition : Inheritance can be explained in middle school with a simple example : "A square is a special rectangle" — the square inherits the properties of the rectangle (4 sides, right angles) but adds the constraint of equal sides. In 12th grade NSI, the `Personne` → `Eleve/Enseignant` hierarchy perfectly illustrates the concept. Encapsulation is pedagogically translated by the black box metaphor : we use an object without knowing its internal workings.

Trainer advice : Multiple inheritance is often a source of confusion. First focus on simple inheritance with concrete examples from the school world. For encapsulation, the `CompteBancaire` example is effective because students intuitively understand that one should not be able to modify a balance directly. Propose a mini-practical where students must add a `ProfesseurPrincipal(Enseignant)` class that inherits from `Enseignant` and adds a `classe` attribute.

Extension : Students can model a complete school management system with a class hierarchy : `Personne` → `Eleve`, `Enseignant`, `PersonnelAdministratif`. Each subclass adds specific attributes and methods. This OOP model will be reused in the ENS2 project and the SQL database.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_2ème_année_AN_04.pdf`

Computer Science Course — ENS Year 2 — Session 04 (EN)

Instructor : BELACEL Madani

Module : Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic Year : 2025-2026

Course sheet — E-learning platform (Moodle)

Reference : benchmark 10 Algerian universities + USTO/Biskra/M'sila practices.

Target audience : ENS 2nd year students — English Department

Prerequisite skills : Algorithms ENS1 (session 05), complexity $O(n)$, $O(n^2)$

Cross-cutting objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare for teaching computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP + SQL Practical 35 % — Integrated project 40 % — Pedagogical lesson plan 25 %

Bibliography : Cormen et al. — Introduction to Algorithms ; Downey — Think Python (recursion chapter)

Specific Learning Objectives

After this session, the student is able to :

- Explain recursion with an example
- Write a simple recursive function (factorial, Fibonacci)
- Implement bubble sort and merge sort (notion)
- Implement binary search
- Compare the complexity of sorting algorithms

Course Plan

1. Recursion : principle and examples
2. Recursion vs iteration
3. Binary search

4. Bubble sort
5. Merge sort
6. Comparison of sorting algorithms

1. Recursion (15 min)

Recursive function : a function that calls itself.

Two essential elements :

- Base case : stopping condition (does not use recursion)
- Recursive call : the function calls itself with a modified parameter

```

flowchart TD
    subgraph Recursion
    A[factorial 5] --> B[5 * factorial 4]
    B --> C[4 * factorial 3]
    C --> D[3 * factorial 2]
    D --> E[2 * factorial 1]
    E --> F[1 base case]
    F --> G[Return 1]
    G --> H[Return 2]
    H --> I[Return 6]
    I --> J[Return 24]
    J --> K[Result 120]
    end

```

Example : factorial

```

def factorielle(n):
    if n <= 1: # Base case
        return 1
    return n * factorielle(n - 1) # Recursive call

print(factorielle(5)) # 120 = 5×4×3×2×1

```

Example : Fibonacci

```

def fibonacci(n):
    if n <= 1: # Base case
        return n
    return fibonacci(n-1) + fibonacci(n-2) # Double call

print(fibonacci(10)) # 55

```

2. Recursion vs iteration (10 min)

```

# Iterative (loop)
def factorielle_iter(n):
    resultat = 1
    for i in range(2, n + 1):
        resultat *= i
    return resultat

# Recursive
def factorielle_rec(n):
    if n <= 1:
        return 1
    return n * factorielle_rec(n - 1)

```

Criterion	Iterative	Recursive
Readability	Simple	Very readable (mathematical)
Performance	Faster	Slower (stacked calls)
Memory	Constant	O(n) call stack
Usage	Simple loops	Divisible problems

3. Binary search (15 min)

Principle : in a sorted array, compare with the middle element, then search left or right.

```
def recherche_dichotomique(tableau, cible):
    debut = 0
    fin = len(tableau) - 1

    while debut <= fin:
        milieu = (debut + fin) // 2
        if tableau[milieu] == cible:
            return milieu # Found
        elif tableau[milieu] < cible:
            debut = milieu + 1 # Search right
        else:
            fin = milieu - 1 # Search left

    return -1 # Not found

# Test
notes = [5, 8, 10, 12, 15, 18, 20]
print(recherche_dichotomique(notes, 15)) # 4
print(recherche_dichotomique(notes, 7)) # -1
```

Complexity : $O(\log n)$ — much faster than linear search $O(n)$.

```
flowchart TD
    subgraph BinarySearch
        A["Sorted array<br/>notes = [5, 8, 10, 12, 15, 18, 20]"] --> B["Target = 15"]
        B --> C["start=0, end=6<br/>mid=3<br/>array[3]=12"]
        C --> D["12 < 15<br/>D[start = mid+1 = 4]"]
        D --> E["start=4, end=6<br/>mid=5<br/>array[5]=18"]
        E --> F["18 > 15<br/>F[end = mid-1 = 4]"]
        F --> G["start=4, end=4<br/>mid=4<br/>array[4]=15"]
        G --> H["15 == 15<br/>H[Found at index 4 ✓]"]
        C --> I["12 > target<br/>I[end = mid-1]"]
        E --> J["18 < target<br/>J[start = mid+1]"]
    end
```

4. Bubble sort (15 min)

Principle : traverse the array, compare adjacent elements, swap them if they are in the wrong order. Repeat until the array is sorted.

```
def tri_bulle(tableau):
    n = len(tableau)
    for i in range(n):
        trie = True
        for j in range(n - i - 1):
            if tableau[j] > tableau[j + 1]:
                tableau[j], tableau[j + 1] = tableau[j + 1], tableau[j]
```

```

trie = False
if trie: # Optimisation : already sorted
    break
return tableau

# Test
notes = [15, 8, 12, 20, 5]
print(tri_bulle(notes)) # [5, 8, 12, 15, 20]

```

Complexity : $O(n^2)$ in the worst case, $O(n)$ if already sorted.

5. Merge sort (15 min)

Principle : divide the array into two, sort each half recursively, then merge.

```

def tri_fusion(tableau):
    if len(tableau) <= 1:
        return tableau

    milieu = len(tableau) // 2
    gauche = tri_fusion(tableau[:milieu])
    droite = tri_fusion(tableau[milieu:])

    return fusionner(gauche, droite)

def fusionner(gauche, droite):
    resultat = []
    i = j = 0
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i])
            i += 1
        else:
            resultat.append(droite[j])
            j += 1
    resultat.extend(gauche[i:])
    resultat.extend(droite[j:])
    return resultat

# Test
notes = [15, 8, 12, 20, 5]
print(tri_fusion(notes)) # [5, 8, 12, 15, 20]

```

Complexity : $O(n \log n)$ in all cases — more efficient than bubble sort.

6. Comparison of sorting algorithms (10 min)

Algorithm	Worst case	Best case	Memory	Stable
Bubble sort	$O(n^2)$	$O(n)$	$O(1)$	Yes
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick sort	$O(n^2)$	$O(n \log n)$	$O(\log n)$	No
Insertion sort	$O(n^2)$	$O(n)$	$O(1)$	Yes

For a teacher :

- Array of 10 grades : bubble sort suffices (simple to understand)

- Class of 30 students : merge sort (faster)
- Database of 10,000 records : merge sort or quick sort

MCQ Self-Assessment

1. What is the base case of the factorial function ?
a) $n == 0$ b) $n \leq 1$ ✓ c) $n == 2$
2. What is the complexity of binary search ?
a) $O(n)$ b) $O(\log n)$ ✓ c) $O(n^2)$
3. Bubble sort has a complexity of :
a) $O(n \log n)$ b) $O(n^2)$ in the worst case ✓ c) $O(1)$
4. Which sorting algorithm is recursive and has $O(n \log n)$ complexity ?
a) Bubble sort b) Merge sort ✓ c) Insertion sort
5. What data structure is required for binary search ?
a) A linked list b) A sorted array ✓ c) A stack

7. Conclusion (5 min)

Recursion allows writing elegant algorithms for divisible problems. Sorting and searching algorithms are fundamental, and their complexity determines their efficiency.

Next lesson : Databases and SQL.

Pedagogical Note ENS

Teaching transposition : Recursion can be introduced in middle school with the example of Russian dolls (matryoshka) or mirrors reflecting each other. In 11th grade NSI, factorial and Fibonacci are classics of the curriculum. Merge sort perfectly illustrates the "divide and conquer" strategy which can be transposed to other disciplines (essay writing, problem solving). Binary search can be presented as the "higher/lower" game that children play intuitively.

Trainer advice : Students often confuse recursion and loops. Insist on the conceptual difference : recursion divides the problem, iteration repeats an action. The recursive/iterative comparison table (section 2) is an excellent summary tool. To assess understanding, ask students to manually trace the execution of `factorielle(4)` on paper.

Extension : Students can implement quick sort and experimentally compare its performance with merge sort on lists of increasing size. They can also create a small program that generates a comparison graph of execution times with `matplotlib`.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_2ème_année_AN_05.pdf`

Computer Science Course — ENS Year 2 — Session 05 (EN)

Instructor : BELACEL Madani

Module : Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic Year : 2025-2026

Course sheet — E-learning platform (Moodle)

Reference : benchmark 10 Algerian universities + USTO/Biskra/M'sila practices.

Target audience : ENS 2nd year students — English Department

Prerequisite skills : Algorithms (session 04), Python OOP (sessions 02-03)

Cross-cutting objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare for teaching computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP + SQL Practical 35 % — Integrated project 40 % — Pedagogical lesson plan 25 %

Bibliography : Connolly & Begg — Database Systems ; Beaulieu — Learning SQL (O'Reilly)

Specific Learning Objectives

After this session, the student is able to :

- Explain the relational model (tables, rows, columns)
- Create a table with SQLite
- Write SELECT, WHERE, JOIN queries
- Insert and update data
- Understand the notion of primary and foreign key

Course Plan

1. Why a database ?
2. Relational model
3. SQLite and Python

4. CREATE TABLE
5. INSERT, SELECT, WHERE
6. Update and delete
7. Foreign keys and JOIN
8. Example : grade management

1. Why a database ? (10 min)

Limitations of a CSV file :

- No strict structure
- No validation (grades > 20 possible)
- Difficult to make links between tables
- No concurrency (two people cannot write at the same time)

Relational database :

- Rigid structure (schema)
- Constraints (keys, types, values)
- Powerful queries (SQL language)
- Data integrity

2. Relational model (10 min)

Table : set of rows (records) and columns (fields).

Table Eleves

id	nom	prenom	groupe
1	Belacel	Ali	ENS2-A
2	Benali	Sara	ENS2-A
3	Cherif	Mohamed	ENS2-B

Vocabulary :

- Primary key : column (or combination) that uniquely identifies each row
- Foreign key : column that references the primary key of another table

```
erDiagram
ELEVES {
    int id PK
    string nom
    string prenom
    string groupe
}
MATIERES {
    int id PK
    string nom
    float coefficient
}
NOTES {
    int id PK
```

```

int eleve_id FK
int matiere_id FK
float valeur
}
ELEVES ||--o{ NOTES : has
MATIERES ||--o{ NOTES : concerns

```

3. SQLite and Python (10 min)

SQLite is a lightweight database, built into Python (no server installation).

```

import sqlite3

# Connection (creates the .db file)
conn = sqlite3.connect("ecole.db")
curseur = conn.cursor()

# Execute a query
curseur.execute("""
CREATE TABLE IF NOT EXISTS eleves (
id INTEGER PRIMARY KEY AUTOINCREMENT,
nom TEXT NOT NULL,
prenom TEXT NOT NULL,
groupe TEXT
)
""")

conn.commit() # Validate
conn.close() # Close

```

4. CREATE TABLE (10 min)

```

conn = sqlite3.connect("ecole.db")
c = conn.cursor()

c.execute("""
CREATE TABLE IF NOT EXISTS matieres (
id INTEGER PRIMARY KEY AUTOINCREMENT,
nom TEXT NOT NULL,
coefficient REAL DEFAULT 1.0
)
""")

c.execute("""
CREATE TABLE IF NOT EXISTS notes (
id INTEGER PRIMARY KEY AUTOINCREMENT,
eleve_id INTEGER NOT NULL,
matiere_id INTEGER NOT NULL,
valeur REAL NOT NULL CHECK(valeur >= 0 AND valeur <= 20),
FOREIGN KEY (eleve_id) REFERENCES eleves(id),
FOREIGN KEY (matiere_id) REFERENCES matieres(id)
)
""")

conn.commit()

```

5. INSERT, SELECT, WHERE (15 min)

INSERT


```
c.execute("INSERT INTO eleves (nom, prenom, groupe) VALUES (?, ?, ?)",
("Belacel", "Ali", "ENS2-A"))
c.execute("INSERT INTO matieres (nom, coefficient) VALUES (?, ?)",
("Computer Science", 3.0))
```

SELECT

```
# All students
c.execute("SELECT * FROM eleves")
for ligne in c.fetchall():
    print(ligne)

# With condition
c.execute("SELECT * FROM eleves WHERE groupe = ?", ("ENS2-A",))
```

```
flowchart LR
    A[Python Client] --> B[SQLite Connection]
    B --> C[Cursor]
    C --> D[execute SQL]
    D --> E{Query type ?}
    E -->|SELECT| F[fetchall / fetchone]
    E -->|INSERT/UPDATE/DELETE| G[commit]
    F --> H[Process results]
    G --> H
    H --> I[close]
```

6. Update and delete (10 min)

```
# UPDATE
c.execute("UPDATE eleves SET groupe = ? WHERE nom = ?",
("ENS2-B", "Belacel"))

# DELETE
c.execute("DELETE FROM notes WHERE valeur < ?", (0,)) # No effect

# Warning : DELETE without WHERE deletes ALL
```

7. Foreign keys and JOIN (15 min)

```
# Insert a grade
c.execute("""
INSERT INTO notes (eleve_id, matiere_id, valeur)
VALUES (?, ?, ?)
""", (1, 1, 15))

# Join : grades + student names + subject names
c.execute("""
SELECT e.nom, e.prenom, m.nom, n.valeur
FROM notes n
JOIN eleves e ON n.eleve_id = e.id
JOIN matieres m ON n.matiere_id = m.id
""")

for ligne in c.fetchall():
    print(f"{ligne[0]} {ligne[1]} – {ligne[2]} : {ligne[3]}")
```

Result :

Belacel Ali – Computer Science : 15.0

8. Complete example : grade report (15 min)

```
import sqlite3

conn = sqlite3.connect("ecole.db")
c = conn.cursor()

# Create tables
c.execute("""
CREATE TABLE IF NOT EXISTS eleves (
id INTEGER PRIMARY KEY AUTOINCREMENT,
nom TEXT, prenom TEXT, groupe TEXT
)
""")
c.execute("""
CREATE TABLE IF NOT EXISTS matieres (
id INTEGER PRIMARY KEY AUTOINCREMENT,
nom TEXT, coefficient REAL
)
""")
c.execute("""
CREATE TABLE IF NOT EXISTS notes (
id INTEGER PRIMARY KEY AUTOINCREMENT,
eleve_id INTEGER, matiere_id INTEGER, valeur REAL,
FOREIGN KEY(eleve_id) REFERENCES eleves(id),
FOREIGN KEY(matiere_id) REFERENCES matieres(id)
)
""")

# Insert data
c.execute("INSERT INTO eleves VALUES (1, 'Ali', 'Belacel', 'ENS2-A')")
c.execute("INSERT INTO matieres VALUES (1, 'Computer Science', 3.0)")
c.execute("INSERT INTO matieres VALUES (2, 'French', 2.0)")
c.execute("INSERT INTO notes VALUES (NULL, 1, 1, 15)")
c.execute("INSERT INTO notes VALUES (NULL, 1, 2, 12)")

# Grade report with average
c.execute("""
SELECT e.nom, e.prenom, m.nom, n.valeur, m.coefficient
FROM notes n
JOIN eleves e ON n.eleve_id = e.id
JOIN matieres m ON n.matiere_id = m.id
WHERE e.id = 1
""")

print("=== Grade Report ===")
total_pondere = 0
total_coef = 0
for ligne in c.fetchall():
    print(f"{ligne[0]} {ligne[1]} - {ligne[2]} : {ligne[3]} (coef {ligne[4]})")
    total_pondere += ligne[3] * ligne[4]
    total_coef += ligne[4]

moyenne = total_pondere / total_coef if total_coef > 0 else 0
print(f"Overall average : {moyenne:.2f}")

conn.commit()
conn.close()
```

MCQ Self-Assessment

1. Which language allows querying a relational database ?

a) HTML b) SQL ✓ c) Python

2. A primary key must be :

a) Unique and not null ✓ b) Optional c) Always of text type

3. Which SQL clause filters results ?

a) SELECT b) WHERE ✓ c) JOIN

4. What does DELETE FROM eLeves do ?

a) Deletes the table b) Deletes all rows ✓ c) Deletes a column

5. A JOIN allows :

a) Combining two tables ✓ b) Sorting results c) Creating a new database

Conclusion

Relational databases and SQL are at the heart of structured data management. SQLite, built into Python, allows quickly prototyping a complete solution (creation, insertion, querying, joining) without server installation. The concepts of primary key, foreign key and join are universal and found in all DBMS (MySQL, PostgreSQL, Oracle).

Next lesson : Web development — HTML, CSS and JavaScript.

Pedagogical Note ENS

Teaching transposition : Databases can be taught in middle and high school without a computer, through unplugged activities :

- Paper file : each student receives a card (table) with fields to fill (name, first name, date of birth). We simulate a SELECT query by sorting the cards.
- Card game : "student" cards and "grade" cards are linked by a paperclip (foreign key). Students must match the cards (JOIN).
- In 10th grade SNT (theme "Structured Data") : make students think about the difference between an unstructured text file and a structured table (spreadsheet). The parallel with a spreadsheet (Excel, LibreOffice Calc) is immediate.
- In 11th grade NSI : SQL is an integral part of the curriculum. The grade report example (section 8) can be directly reused in class.

Trainer advice : Ask ENS students to prepare a 45-minute pedagogical lesson plan on databases for an 8th or 10th grade class, incorporating an unplugged activity and a digital activity. This promotes teaching transposition, a key skill of the teaching profession.

Extension : Students can create a mini library management application (books, authors, loans) with SQLite and Python, and think about useful queries for a teacher (grade management, absences, groups).

Templates : [Template/PG-fr.docx](#) | [Template/Presentation_BELACEL_V3.pptx](#)

PDF : [Cours_Informatique_ENS_2ème_année_AN_06.pdf](#)

Computer Science Course — ENS Year 2 — Session 06 (EN)

Instructor : BELACEL Madani

Module: Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic year : 2025-2026

Course Sheet — E-learning Platform (Moodle)

Reference framework : benchmark of 10 Algerian universities + practices USTO/Biskra/M'sila.

Target audience : ENS Year 2 Students — English Department

Prerequisite skills : Networks (ENS1 session 07), Python files (ENS1 session 09)

Cross-curricular objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare the teaching of computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP Lab + SQL 35 % — Integrated Project 40 % — Pedagogical Session Sheet 25 %

Bibliography : MDN Web Docs (developer.mozilla.org) ; Duckett — HTML & CSS

Specific Learning Objectives

At the end of this session, the student is able to :

- Create a structured HTML page (head, body, sections)
- Apply CSS styles (colors, fonts, layout)
- Add a simple JavaScript script (user interaction)
- Explain the HTTP request/response cycle
- Understand the role of Web forms

Course Plan

1. Web Architecture (client-server)
2. HTML: page structure
3. CSS: styling

4. JavaScript: interaction
5. Web Forms
6. Deployment and hosting
7. Mini-project: teacher personal page

1. Web Architecture (10 min)

Client-server : the browser (client) sends an HTTP request to the server, which responds with the content.

```
sequenceDiagram
    participant Client as Browser
    participant Serveur as Web Server
    participant BD as Database

    Client->>Serveur: GET /index.html
    Serveur-->>Client: 200 OK [HTML]
    Client->>Serveur: GET /style.css
    Serveur-->>Client: 200 OK [CSS]
    Client->>Serveur: POST /send (form)
    Serveur->>BD: INSERT INTO messages
    BD-->>Serveur: OK
    Serveur-->>Client: 301 Redirect /thankyou
    Client->>Serveur: GET /thankyou
    Serveur-->>Client: 200 OK [HTML]
```

URL : <https://www.ens-mostaganem.dz/departements/english>

- https : secure protocol
- www.ens-mostaganem.dz : domain name
- /departements/english : path

2. HTML (20 min)

HTML (HyperText Markup Language) uses tags to structure the content.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My Portfolio – Teacher</title>
</head>
<body>
<header>
<h1>BELACEL Madani</h1>
<nav>
<a href="#courses">Courses</a>
<a href="#research">Research</a>
<a href="#contact">Contact</a>
</nav>
</header>

<main>
<section id="courses">
<h2>My courses</h2>
<ul>
<li>Computer Science ENS Year 1</li>
<li>Computer Science ENS Year 2</li>
```

```

<li>ICT – Bachelor</li>
</ul>
</section>

<section id="research">
<h2>Research</h2>
<table>
<tr><th>Year</th><th>Publication</th></tr>
<tr><td>2025</td><td>ICT in Algeria</td></tr>
</table>
</section>
</main>

<footer>
<p>© 2026 BELACEL Madani – ENS Mostaganem</p>
</footer>
</body>
</html>

```

Semantic tags : <header>, <nav>, <main>, <section>, <footer> (better SEO and accessibility).

```

graph TD
A[html] --> B[head]
A --> C[body]
B --> D[meta charset]
B --> E[title]
C --> F[header]
C --> G[main]
C --> H[footer]
F --> I[h1]
F --> J[nav]
J --> K[a href]
J --> L[a href]
G --> M[section id=courses]
G --> N[section id=research]
M --> O[h2]
M --> P[ul]
P --> Q[li]
P --> R[li]
N --> S[table]
S --> T[tr]
T --> U[th]
T --> V[th]
H --> W[p]

style A fill:#003366,color:#fff
style B fill:#003366,color:#fff
style C fill:#003366,color:#fff

```

3. CSS (15 min)

CSS (Cascading Style Sheets) controls the appearance.

```

/* style.css */
body {
font-family: 'Segoe UI', Arial, sans-serif;
margin: 0;
padding: 20px;
background-color: #f5f5f5;
color: #333;
}

```

```

header {
background-color: #003366;
color: white;
padding: 20px;
text-align: center;
}

nav a {
color: white;
margin: 0 10px;
text-decoration: none;
}

nav a:hover {
text-decoration: underline;
}

table {
border-collapse: collapse;
width: 100%;
}

th, td {
border: 1px solid #ddd;
padding: 8px;
text-align: left;
}

th {
background-color: #003366;
color: white;
}

```

Integration : `<link rel="stylesheet" href="style.css">` in the `<head>`.

4. JavaScript (15 min)

JavaScript enables client-side interactivity.

```

<!DOCTYPE html>
<html>
<body>
<h2>Average Calculator</h2>
<input type="number" id="grade1" placeholder="Grade 1">
<input type="number" id="grade2" placeholder="Grade 2">
<input type="number" id="grade3" placeholder="Grade 3">
<button onclick="calculate()">Calculate</button>
<p id="result"></p>

<script>
function calculate() {
let n1 = parseFloat(document.getElementById("grade1").value);
let n2 = parseFloat(document.getElementById("grade2").value);
let n3 = parseFloat(document.getElementById("grade3").value);
let average = (n1 + n2 + n3) / 3;
document.getElementById("result").innerHTML =
"Average: " + average.toFixed(2);
}
</script>
</body>
</html>

```

5. Forms (15 min)

```
<form action="/send" method="POST">
<label for="name">Name :</label>
<input type="text" id="name" name="name" required>

<label for="email">Email :</label>
<input type="email" id="email" name="email">

<label for="message">Message :</label>
<textarea id="message" name="message" rows="5"></textarea>

<button type="submit">Send</button>
</form>
```

Input types : text, email, number, date, file, password, checkbox, radio.

6. Deployment (10 min)

Free options to host a teacher website :

- GitHub Pages : username.github.io
- Netlify : automatic deployment from Git
- Google Sites : no code (WYSIWYG)
- Moodle : integrated course space

7. Mini-project (10 min)

Create a personal web page for your future teaching career :

1. HTML structure (bio, courses, contact)
2. CSS style (ENS colors: dark blue #003366)
3. Contact form
4. Link to GitHub repository

MCQ Self-Assessment

1. What does HTML stand for ?
a) HyperText Markup Language ✓ b) High Tech Modern Language c) Home Tool Markup Language
2. Which HTML tag creates a hyperlink ?
a) <link> b) <a> ✓ c) <href>
3. Which language controls the appearance of a Web page ?
a) HTML b) CSS ✓ c) JavaScript
4. Which HTML input type is used for passwords ?
a) text b) password ✓ c) hidden

5. What is an HTTP GET request ?

a) Sends data to the server via the URL ✓ b) Modifies the database c) Establishes a permanent connection

8. Conclusion (5 min)

HTML structures, CSS styles, JavaScript animates. These three technologies are the foundation of the Web. A modern teacher knows how to create and publish content online.

Next course : Git and collaboration.

Pedagogical Note ENS

Teaching transposition : The Web can be taught in middle school through the creation of simple HTML pages. In SNT Seconde (theme "Web"), students learn to distinguish client and server, understand the role of the browser, and inspect the source code of a page. The "teacher personal page" mini-project activity can be adapted for 9th-grade students by having them create an HTML page about a career that interests them. The client-server sequence can be simulated through a role-play: one student is the server, another is the client.

Trainer advice : Students often struggle with the HTML/CSS separation. Emphasize the single responsibility principle: HTML for content, CSS for presentation. An effective exercise consists of providing the same HTML and asking students to apply different CSS to produce completely different visual renderings. This illustrates the power and flexibility of CSS.

Extension : Students can create a complete mini-teacher website with multiple pages (home, courses, contact), a responsive navigation menu (media queries), and a functional contact form using a service like Formspree or a simple Flask backend. They can also add a "project portfolio" page with screenshots and GitHub links.

Templates : [Template/PG-fr.docx](#) | [Template/Presentation_BELACEL_V3.pptx](#)

PDF : [Cours_Informatique_ENS_2ème_année_AN_07.pdf](#)

Computer Science Course — ENS Year 2 — Session 07 (EN)

Instructor : BELACEL Madani

Module: Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic year : 2025-2026

Course Sheet — E-learning Platform (Moodle)

Reference framework : benchmark of 10 Algerian universities + practices USTO/Biskra/M'sila.

Target audience : ENS Year 2 Students — English Department

Prerequisite skills : Basic terminal usage (ENS1 session 03), Python projects

Cross-curricular objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare the teaching of computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP Lab + SQL 35 % — Integrated Project 40 % — Pedagogical Session Sheet 25 %

Bibliography : Chacon & Straub — Pro Git (git-scm.com) ; Atlassian — Git Tutorials

Specific Learning Objectives

At the end of this session, the student is able to :

- Initialize a Git repository
- Add and commit files
- Create and merge branches
- Push/pull to GitHub
- Write a professional README
- Use .gitignore

Course Plan

1. Why Git ?
2. Installation and configuration

3. Basic commands (init, add, commit, status)
4. Branches (branch, merge)
5. GitHub: remote, push, pull
6. README and documentation
7. Best practices for teachers

1. Why Git ? (10 min)

Problem : "TD_final.docx", "TD_final2.docx", "TD_version_final.docx"...

Solution : Git — version control system.

Advantages :

- Complete history of modifications
- Teamwork without conflicts
- Error rollback possible
- Cloud backup (GitHub)
- Code portfolio for teachers

2. Installation and configuration (10 min)

Installation

```
# Check if Git is installed
git --version

# Download: git-scm.com
```

Initial configuration

```
git config --global user.name "Madani BELACEL"
git config --global user.email "m.belacel@ens-mostaganem.dz"
git config --global core.editor "code --wait"
```

3. Basic commands (20 min)

```
# Initialize a repository in the project folder
cd projet-ens2
git init

# View status
git status

# Add a file to tracking (stage)
git add cours_web.html
git add . # Add entire folder

# Save a version (commit)
git commit -m "Add ENS2 web course page"

# View history
git log --oneline
```

File lifecycle :

```

flowchart LR
  A[Working Directory<br/>Modified files] -->|git add| B[Staging Area<br/>Indexed files]
  B -->|git commit| C[Repository<br/>Saved version]
  C -->|git checkout| A
  C -->|git push| D[Remote<br/>GitHub]
  D -->|git pull| C
  D -->|git clone| E[Other machine]

  style A fill:#f9d71c,stroke:#333
  style B fill:#f9a825,stroke:#333
  style C fill:#4caf50,stroke:#333
  style D fill:#2196f3,stroke:#333

```

4. Branches (20 min)

Branch : parallel version of the project. Allows working on a feature without affecting the main branch.

```

# List branches
git branch

# Create a branch
git branch quiz-feature

# Switch branch
git checkout quiz-feature

# Create and switch (in one command)
git checkout -b new-feature

```

Merge

```

# Switch back to main
git checkout main

# Merge the working branch
git merge quiz-feature

# Delete the branch after merging
git branch -d quiz-feature

gitGraph
  commit id: "Initial commit"
  commit id: "HTML Structure"
  branch quiz-feature
  checkout quiz-feature
  commit id: "Add HTML quiz"
  commit id: "CSS quiz style"
  checkout main
  commit id: "Navbar fix"
  merge quiz-feature id: "Merge quiz"
  commit id: "README"
  branch unit-test
  checkout unit-test
  commit id: "Test model.py"
  checkout main
  merge unit-test id: "Merge tests"
  commit id: "v1.0"

```

Conflicts : when two branches modify the same line. Git shows both versions, the user must choose.

5. GitHub (15 min)

Create a remote repository

1. Create an account on github.com
2. Click on New repository
3. Copy the URL : `https://github.com/user/projet.git`

Link local → remote repository

```
git remote add origin https://github.com/user/projet.git
git push -u origin main
```

Update

```
git push # Send your commits
git pull # Fetch others' commits
```

GitHub Pages : Settings → Pages → branch: main → free website.

6. README and documentation (10 min)

The README.md file is the repository's home page (Markdown).

```
# ENS2 Project – Grade Management

## Description
Web application for managing student grades.
Developed as part of the ENS Year 2 Computer Science module.

## Features
- Add/modify/delete students
- Grade entry with validation (0-20)
- Automatic average calculation
- CSV export

## Technologies
- Python 3 + SQLite
- HTML/CSS/JavaScript
- Git + GitHub

## Installation

git clone https://github.com/user/projet-ens2.git
cd projet-ens2
python main.py

## Author
BELACEL Madani – ENS Mostaganem
```

7. .gitignore (5 min)

.gitignore file : ignores files that should not be versioned.

```
__pycache__/  
*.pyc  
*.db  
.vscode/  
.env  
.DS_Store  
node_modules/
```

8. Best practices (5 min)

1. Frequent and atomic commits (one modification = one commit)
2. Clear commit messages : `git commit -m "Add average calculation function"`
3. Never commit : passwords, API keys, student data
4. Always pull before starting to work
5. Use branches for experiments
6. Add a README from the start

MCQ Self-Assessment

1. Which Git command initializes a new repository ?
a) `git start` b) `git init` ✓ c) `git new`
2. Which command adds a file to the staging area ?
a) `git commit` b) `git add` ✓ c) `git push`
3. A Git branch allows you to :
a) Permanently delete files b) Work in parallel without affecting the main branch ✓ c) Speed up commits
4. Which command sends commits to GitHub ?
a) `git push` ✓ b) `git pull` c) `git clone`
5. What does the `.gitignore` file contain ?
a) The list of contributors b) Files not to be versioned ✓ c) The last commit message

9. Conclusion (5 min)

Git is the standard tool of modern development. It allows you to keep a history, collaborate, and showcase your work (GitHub portfolio).

Next course : Didactics of computer science — teaching in middle/high school.

Pedagogical Note ENS

Teaching transposition : Git can be introduced as early as 9th grade through an unplugged activity: each student receives a sheet (file) and must go through "commits" at each significant modification. Branches are represented by different colored sheets, and merging by a stapler that binds the sheets together. In NSI Première, versioning is part of the

curriculum and can be presented as a development logbook. The parallel with the "versions" of an assignment submitted by a student is revealing.

Trainer advice : Students often struggle with the concepts of branches and merging. Start with a simple scenario: "You work on the project alone, you create a branch for a feature, you merge it." Once this cycle is mastered, move on to collaborative work with two people on the same repository. Emphasize the importance of clear commit messages — this is a professional skill.

Extension : Students can create a Git repository from scratch for their teacher portfolio, using GitHub Pages to publish it. They can also intentionally experiment with merge conflicts (two students modify the same file) and learn to resolve them — this skill is highly valued in the professional environment.

Templates : [Templatte/PG-fr.docx](#) | [Templatte/Presentation_BELACEL_V3.pptx](#)

PDF : [Cours_Informatique_ENS_2ème_année_AN_08.pdf](#)

Computer Science Course — ENS Year 2 — Session 08 (EN)

Instructor : BELACEL Madani

Module: Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic year : 2025-2026

Course Sheet — E-learning Platform (Moodle)

Reference framework : benchmark of 10 Algerian universities + practices USTO/Biskra/M'sila.

Target audience : ENS Year 2 Students — English Department

Prerequisite skills : All previous sessions (module synthesis)

Cross-curricular objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare the teaching of computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP Lab + SQL 35 % — Integrated Project 40 % — Pedagogical Session Sheet 25 %

Bibliography : Eduscol — Teaching programs ; Scratch MIT ; Teacher's guide — SNT

Specific Learning Objectives

At the end of this session, the student is able to :

- Know the official computer science programs in middle/high school
- Design a Scratch session for middle school students
- Use Python as a pedagogical tool
- Assess digital skills (PIX/C2i)
- Write a pedagogical session sheet

Course Plan

1. Official programs (middle school, high school, SNT, NSI)
2. Scratch as introduction
3. Pedagogical Python

4. Digital skills assessment
5. Session sheet: model
6. Artificial intelligence in the classroom
7. Digital usage charter

1. Official programs (15 min)

Middle School (Cycle 4)

- Algorithms and programming (Scratch)
- Internet and data
- Connected objects
- Volume : ~15h in 9th grade

High School — SNT (Seconde)

- Digital Sciences and Technology
- Internet, Web, data, AI, geolocation
- Volume : 1h30/week

High School — NSI (Première/Terminale)

- Python programming (advanced)
- Architecture, networking, databases
- Volume : 4h/week

```
mindmap
  root((Programs<br/>Computer Science))
    Middle School Cycle 4
      Scratch Algorithms
      Internet
      Data
      Connected objects
      Volume: 15h 9th grade
    High School SNT Seconde
      Internet & Web
      Structured data
      Artificial Intelligence
      Geolocation
      Volume: 1h30/week
    High School NSI
      ère: Python, Web, Architecture
      Tale: Recursion, DB, Project
      Volume: 4h/week
    Cross-curricular skills
    PIX (France)
    C2i (Algeria)
    Digital ethics
    Usage charter
```

2. Scratch (20 min)

Scratch (scratch.mit.edu) : visual block programming — ideal for middle school.

Concepts taught :

- Movement, looks, sound blocks
- Loops (repeat, forever)
- Conditions (if, if... then)
- Variables and lists
- Events (when flag clicked, when key pressed)

Example exercise : walking cat

1. When flag clicked → move 10 steps
2. If space key pressed → say "Hello !" for 2 seconds
3. Bounce if edge reached

Recommended progression :

1. Simple blocks (movement)
2. Loops and conditions
3. Variables (score, timer)
4. Final project (pong game, quiz)

```

flowchart LR
    subgraph Middle school
        A[6e-5e<br/>Scratch Discovery] --> B[4e<br/>Advanced blocks]
        B --> C[3e<br/>Introduction Python]
    end
    subgraph High school
        C --> D[2de SNT<br/>Simple Python]
        D --> E[1ère NSI<br/>Advanced Python]
        E --> F[Tale NSI<br/>Complex projects]
    end
    subgraph ENS
        F --> G[ENS1<br/>Python review]
        G --> H[ENS2<br/>OOP, SQL, Web, Git]
    end
    style Middle school fill:#e8f5e9,stroke:#2e7d32
    style High school fill:#e3f2fd,stroke:#1565c0
    style ENS fill:#fff3e0,stroke:#e65100

```

3. Pedagogical Python (15 min)

When to move from Scratch to Python ?

- Scratch : 6e-4e (discovery, logic)
- Python : 3e-2de (first textual programming)

Approach :

```

# Python version of a Scratch exercise
# Say "Hello" when you click on the flag

import turtle

t = turtle.Turtle()

# Move forward 50 steps
t.forward(50)

```

```
# Turn right
t.right(90)
t.forward(50)

turtle.done()
```

Pedagogical libraries :

- turtle : drawing, movement (Scratch transition)
- random : games, simulations
- tkinter : simple graphical interfaces

4. Skills assessment (10 min)

PIX (France)

Digital skills assessment platform (16 skills).

- Search engine, email, files, spreadsheet, programming
- Official certification

C2i (Algeria)

Computer and Internet Certificate — level 1 (Bachelor).

Assessed skills :

- B1 : Information search and monitoring
- B2 : Digital identity and GDPR
- B3 : Document production
- B4 : Remote collaboration

5. Pedagogical session sheet (15 min)

Model for designing a computer science session for middle school :

```
# Session Sheet – Middle School Computer Science

**Level **: 9th grade
**Theme **: Algorithms and programming
**Duration **: 45 minutes

## Objectives
- Understand the concept of a loop
- Use a "repeat" loop in Scratch

## Prerequisites
- Know how to move a sprite
- Know "move" and "turn" blocks

## Procedure
| Phase | Duration | Activity |
|-----|-----|-----|
| Discovery | 10 min | Presentation of the magic square |
| Research | 15 min | Create a square by repetition |
| Sharing | 10 min | Compare solutions |
```

| Synthesis | 5 min | Written summary: "repeat 4 times" |
| Assessment | 5 min | Kahoot quiz on loops |

Resources

- Student worksheet
- Starting Scratch project
- Assessment grid

Review

- Strengths :
- Difficulties encountered :
- Remediation :

6. AI in the classroom (10 min)

Opportunities :

- Grading assistant (save time)
- Personalized exercise generation
- Translation and adaptation
- Help with writing lesson sheets

Limits and precautions :

- Verify results (AI can be wrong)
- Do not enter personal student data
- Raise student awareness about plagiarism
- Cite AI usage

Golden rule : AI is a tool, not a substitute for the teacher's judgment.

7. Digital usage charter (5 min)

Elements to include in a classroom charter :

1. Use only your institutional account
2. Do not share your passwords
3. Respect classmates' privacy
4. Report any inappropriate content
5. Cite your sources (images, texts)
6. Turn off your phone in class (unless for pedagogical use)
7. Ask permission before recording/filming

MCQ Self-Assessment

1. At which level is Scratch mainly taught ?
a) High school b) Middle school (Cycle 4) ✓ c) University

2. What is the weekly volume of the NSI option in Première ?
a) 1h30/week b) 4h/week ✓ c) 6h/week
3. Which Python library allows a pedagogical transition from Scratch ?
a) numpy b) turtle ✓ c) django
4. PIX is an assessment platform for :
a) Programming skills only b) Overall digital skills ✓ c) Foreign languages
5. What is the golden rule regarding AI in the classroom ?
a) AI replaces the teacher b) AI is a tool, not a substitute for the teacher's judgment ✓ c) AI is banned in education

8. Conclusion (5 min)

As a future teacher, you must master both computer science concepts and their didactics. Scratch for middle school, Python for high school, and a solid digital ethics for all.

Next course : ENS2 project design — Web application or Python OOP module.

Pedagogical Note ENS

Teaching transposition : This session is itself an example of teaching transposition: advanced computer science concepts (OOP, SQL, Web) are "transposed" into accessible activities for secondary education. The Scratch → Python progression (section 3) is at the heart of current programs. The proposed session sheet model can be used for any level. Student-teachers must understand that the difficulty is not mastering computer science, but teaching it to beginners — this is the whole challenge of didactics.

Trainer advice : Organize a micro-teaching session: each student prepares and leads a 10-minute sequence on a computer science concept (loop, condition, variable) in front of the group. Film the presentations for constructive feedback. Peer assessment (criteria-based grid) strengthens professionalization. Do not forget to raise awareness about classroom management: what to do when a student is stuck on an exercise? How to differentiate instruction ?

Extension : Students can design a complete sequence of 3 sessions on a computer science topic (creating a Scratch game, introduction to Python with Turtle, discovering the Web with HTML) including student worksheets, assessment grids, and written summaries. These productions will be reused during the practical internship.

Templates : [Templatte/PG-fr.docx](#) | [Templatte/Presentation_BELACEL_V3.pptx](#)

PDF : [Cours_Informatique_ENS_2ème_année_AN_09.pdf](#)

Computer Science Course — ENS Year 2 — Session 09 (EN)

Instructor : BELACEL Madani

Module: Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic year : 2025-2026

Course Sheet — E-learning Platform (Moodle)

Reference framework : benchmark of 10 Algerian universities + practices USTO/Biskra/M'sila.

Target audience : ENS Year 2 Students — English Department

Prerequisite skills : All module sessions (OOP, SQL, Web, Git, didactics)

Cross-curricular objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare the teaching of computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP Lab + SQL 35 % — Integrated Project 40 % — Pedagogical Session Sheet 25 %

Bibliography : Python, Flask, SQLite, GitHub documentation

Specific Learning Objectives

At the end of this session, the student is able to :

- Plan a complete software project
- Design a Web application architecture
- Structure code into separate modules
- Use Flask to create a simple Web application
- Apply Git throughout development

Course Plan

1. ENS2 project specifications
2. Application architecture
3. Technical choices

4. File structure
5. Guided development: Flask skeleton
6. Database connection
7. Planning with GitHub Projects

1. Specifications (10 min)

ENS2 Project — Grade management application for teachers

Mandatory features

- Add/modify/delete students
- Add grades with validation (0-20)
- Automatic average calculation
- Display a grade report (table)
- CSV export

Optional features

- Charts (matplotlib)
- Authentication (login/password)
- Light/dark theme
- Multilingual mode (EN/FR/AR)

Technical constraints

- Python 3 + SQLite
- Flask (Web) or console interface + HTML export
- Git + GitHub (public repository)
- Complete README.md
- Associated pedagogical session sheet

2. Architecture (10 min)

```
projet-ens2/
├── app.py # Flask entry point
├── modele.py # OOP Classes (Student, Grade, Subject)
├── database.py # SQLite connection and queries
├── templates/ # HTML templates (Jinja2)
│   ├── index.html
│   ├── eleves.html
│   └── releve.html
├── static/ # CSS, JS, images
│   ├── style.css
│   └── script.js
├── donnees/ # Data files
│   └── ecole.db
├── tests/ # Unit tests
│   └── test_modele.py
├── requirements.txt # Dependencies
└── .gitignore
```

└─ README.md

```
graph TD
    subgraph View [View - HTML Templates]
        A[index.html] --> B[eleves.html]
        A --> C[releve.html]
    end
    subgraph Controller [Controller - Flask]
        D[app.py] --> E[Routes]
        E --> F[/]
        E --> G[/eleves]
        E --> H[/ajouter]
        E --> I[/releve]
    end
    subgraph Model [Model - Python + SQLite]
        J[modele.py] --> K[OOP Classes]
        K --> L[Student]
        K --> M[Grade]
        L --> N[database.py]
        M --> N
        N --> O[(school.db)]
    end
    D --> A
    D --> B
    D --> C
    J --> N

    style View fill:#e3f2fd,stroke:#1565c0
    style Controller fill:#fff3e0,stroke:#e65100
    style Model fill:#e8f5e9,stroke:#2e7d32
```

Principe : separation of responsibilities (model, view, controller — MVC).

3. Technical choices (10 min)

Component	Technology	Justification
Language	Python 3	Simple, all-in-one
Web Framework	Flask	Lightweight, easy to learn
Database	SQLite	Single file, no server
Templates	Jinja2 (built-in Flask)	Simple syntax
CSS	Bootstrap 5	Fast, responsive
Versioning	Git + GitHub	Portfolio, collaboration
Deployment	GitHub Pages or PythonAnywhere	Free

4. Guided code — Flask (20 min)

```
# app.py
from flask import Flask, render_template, request, redirect
from modele import Eleve, Note
from database import init_db, ajouter_eleve, get_eleves

app = Flask(__name__)

@app.route("/")
```



```

def index():
    return render_template("index.html")

@app.route("/eleves")
def liste_eleves():
    eleves = get_eleves()
    return render_template("eleves.html", eleves=eleves)

@app.route("/ajouter", methods=["POST"])
def ajouter():
    nom = request.form["nom"]
    prenom = request.form["prenom"]
    groupe = request.form["groupe"]
    ajouter_eleve(nom, prenom, groupe)
    return redirect("/eleves")

if __name__ == "__main__":
    init_db()
    app.run(debug=True)

```

5. OOP Model (15 min)

```

# modele.py
class Eleve:
    def __init__(self, id, nom, prenom, groupe):
        self.id = id
        self.nom = nom
        self.prenom = prenom
        self.groupe = groupe
        self.notes = []

    def moyenne(self):
        if not self.notes:
            return 0.0
        return sum(n.valeur for n in self.notes) / len(self.notes)

    def ajouter_note(self, matiere, valeur):
        if 0 <= valeur <= 20:
            self.notes.append(Note(self.id, matiere, valeur))

class Note:
    def __init__(self, eleve_id, matiere, valeur):
        self.eleve_id = eleve_id
        self.matiere = matiere
        self.valeur = valeur

```

6. Database (10 min)

```

# database.py
import sqlite3

DB = "donnees/ecole.db"

def init_db():
    conn = sqlite3.connect(DB)
    c = conn.cursor()
    c.execute("""
CREATE TABLE IF NOT EXISTS eleves (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,

```

```

prenom TEXT NOT NULL,
groupe TEXT
)
"""
c.execute("""
CREATE TABLE IF NOT EXISTS notes (
id INTEGER PRIMARY KEY AUTOINCREMENT,
eleve_id INTEGER NOT NULL,
matiere TEXT NOT NULL,
valeur REAL NOT NULL CHECK(valeur >= 0 AND valeur <= 20),
FOREIGN KEY(eleve_id) REFERENCES eleves(id)
)
""")
conn.commit()
conn.close()

def get_eleves():
conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row
c = conn.cursor()
c.execute("SELECT * FROM eleves")
eleves = [dict(row) for row in c.fetchall()]
conn.close()
return eleves

```

7. Planning (5 min)

Use GitHub Projects or a Kanban board :

Sprint	Deadline	Deliverable
Sprint 1	D+7	Database + OOP model
Sprint 2	D+14	Flask interface (HTML/CSS)
Sprint 3	D+21	Complete features
Sprint 4	D+28	Tests + Deployment
Sprint 5	D+30	Defense

```

gantt
title ENS2 Project Planning
dateFormat YYYY-MM-DD
axisFormat %d/%m

section Sprint 1
OOP Model + SQLite DB :done, s1, 2026-02-01, 7d

section Sprint 2
Flask Interface + Templates :done, s2, after s1, 7d

section Sprint 3
CRUD Features :active, s3, after s2, 7d

section Sprint 4
Tests + Deployment :s4, after s3, 7d

section Sprint 5
Defense :milestone, after s4, 1d

```

MCQ Self-Assessment

1. Which Web framework is used in the ENS2 project ?
a) Django b) Flask ✓ c) FastAPI
2. Which database management system is used ?
a) MySQL b) PostgreSQL c) SQLite ✓
3. Which architectural model does the project follow ?
a) MVC (Model-View-Controller) ✓ b) MVP c) Microservices
4. Which feature is mandatory in the project ?
a) Authentication b) CSV export ✓ c) Multilingual mode
5. How many sprints are planned ?
a) 3 b) 5 ✓ c) 7

8. Conclusion (5 min)

The ENS2 project is the culmination of the module. It mobilizes all the skills acquired: OOP, databases, Web, Git, and didactics.

Next course : ENS2 project defense — presentation and demonstration.

Pedagogical Note ENS

Teaching transposition : The project methodology presented here (specifications, architecture, sprints) can be transposed to middle/high school in the form of a Scratch or Python project. The important thing is to follow the same approach: 1) define the need, 2) design the solution, 3) implement by iterations, 4) test, 5) present. In NSI Terminale, the project is a baccalaureate exam — this session thus directly prepares future teachers to support their students.

Trainer advice : The ENS2 project is the moment of truth for the module. Ensure that each student has a functional minimal project before aiming for advanced features ("Make it work, then make it better"). Encourage collaboration but verify individual contributions via Git (each student must commit). Schedule a peer code review session before the defense.

Extension : Students can extend the project with advanced features: CSV import (to load an existing class), grade distribution chart generation, sending grade reports by email, or an administration interface with login. These extensions can be presented as "bonuses" during the defense and valued in the grade.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_2ème_année_AN_10.pdf`

Computer Science Course — ENS Year 2 — Session 10 (EN)

Instructor : BELACEL Madani

Module: Computer Science

Level : ENS — Year 2 — English Department

Duration : 1h30

Academic year : 2025-2026

Course Sheet — E-learning Platform (Moodle)

Reference framework : benchmark of 10 Algerian universities + practices USTO/Biskra/M'sila.

Target audience : ENS Year 2 Students — English Department

Prerequisite skills : ENS2 project developed (session 09), Git, OOP, SQL, Flask

Cross-curricular objectives :

- Apply object-oriented programming (classes, inheritance, encapsulation) in Python
- Design advanced algorithms (sorting, searching, recursion) and estimate their complexity
- Model data with the relational model and basic SQL queries
- Understand Web architecture (HTML, CSS, JavaScript intro, HTTP)
- Use Git and collaborative work methods (branches, merge)
- Prepare the teaching of computer science in middle/high school (Scratch, pedagogical Python)

Assessment method : OOP Lab + SQL 35 % — Integrated Project 40 % — Pedagogical Session Sheet 25 %

Bibliography : MDN Web Docs ; Flask Documentation ; Git Book

Specific Learning Objectives

At the end of this session, the student is able to :

- Present a complete software project
- Demonstrate the developed features
- Explain technical and architectural choices
- Show the Git repository with history
- Present the associated pedagogical session sheet

Course Plan

1. Detailed evaluation criteria
2. Presentation structure (10 min)
3. Live demo

4. Technical questions
5. Pedagogical session sheet
6. ENS2 module review
7. Next steps (internship, portfolio)

1. Evaluation criteria (5 min)

Criterion	Weight	Detail
Features	30 %	CRUD students/grades, average, CSV export
OOP Code	20 %	Classes, inheritance, methods
Database	15 %	SQLite, tables, queries, joins
Web Interface	10 %	Flask, HTML/CSS, templates
Git	10 %	Regular commits, branches, README
Pedagogical sheet	10 %	Middle/high school session with Scratch
Oral presentation	5 %	Clarity, structure, demo

2. Presentation structure (10 min)

Each student has 10 minutes :

```
mindmap
  root((ENS2 Project<br/>10 min Defense))
    Presentation
    Objective
    Target audience
    School context
    Architecture
    MVC diagram
    Technologies
    Database
    Live demo
    Add student
    Grade entry
    Grade report
    CSV export
    Code
    Student OOP Class
    SQL queries
    Flask routes
    HTML templates
    Versioning
    Git history
    Branches
    README
    .gitignore
    Pedagogy
    Middle school session sheet
    Scratch progression
    Assessment
    Conclusion
    Skills review
    Perspectives
    Questions
```

Minute	Content
0-1	Project presentation (objective, target audience)
1-3	Architecture (diagram, technologies)
3-5	Live demo (add student, grade entry, report)
5-7	Code: Student class, SQL query, Flask route
7-8	Git: history, branches, README
8-9	Associated pedagogical session sheet
9-10	Conclusion + questions

3. Live demo — typical scenario (20 min)

Demonstration scenario :

1. Home — Home page with presentation
2. Add student — Enter "Belacel", "Ali", "ENS2-A"
3. Add grades — Computer Science: 15, French: 12, English: 18
4. Average calculation — Verify that the average displays correctly
5. Grade report — Display the complete table
6. CSV export — Download the file and open it in Excel
7. GitHub — Show the repository, commits, README

Errors to anticipate :

- Adding a grade > 20 → expected error message
- Deleting a student → confirmation
- Navigation → responsive (mobile/desktop)

4. Possible technical questions (15 min)

Prepare answers to these questions :

OOP :

- Why did you choose inheritance in your model ?
- What is the difference between a class attribute and an instance attribute ?
- How did you implement encapsulation ?

Database :

- Why SQLite rather than MySQL/PostgreSQL ?
- What is the primary key ? Why AUTOINCREMENT ?
- How did you handle the $0 \leq \text{grade} \leq 20$ constraint ?

Web :

- Explain the request/response cycle in Flask

- How do you pass data from Python to HTML ?
- What is a route in Flask ?

Git :

- Show a commit where you fixed a bug
- What is the difference between merge and rebase ?
- How do you handle a merge conflict ?

Pedagogy :

- How would you adapt this project for 9th grade students ?
- What written summary do you propose for students ?
- How do you assess PIX skills ?

5. Pedagogical session sheet (15 min — individual work)

Write a 45-minute session sheet for middle school, related to the project.

Proposed topic : Discovering programming with Scratch — Creating an interactive quiz

Use the model from session 08 :

```
# Session Sheet – Interactive quiz with Scratch

**Level **: 9th grade (Cycle 4)
**Theme **: Algorithms and programming
**Duration **: 45 minutes

## Objectives
- Use a variable (score)
- Implement a condition (if the answer is correct)
- Create a game loop

## Procedure
1. Discovery (5 min): show the completed quiz
2. Variables (10 min): create "score" variable
3. Questions (15 min): add 3 questions with conditions
4. Test (10 min): play and debug
5. Review (5 min): written summary

## Assessment
- Functional final quiz (3 questions minimum)
- Score variable that increments
- Feedback message (well done/try again)
```

6. ENS2 module review (10 min)

```
quadrantChart
title Acquired skills – ENS2 Computer Science Module
x-axis "Technical" --> "Pedagogical"
y-axis "Basic" --> "Advanced"
quadrant-1 "Pedagogical expertise"
quadrant-2 "Technical expertise"
quadrant-3 "Fundamentals"
quadrant-4 "In development"
Python OOP: [0.75, 0.85]
Algorithms: [0.80, 0.70]
Databases: [0.70, 0.75]
```

Web Development: [0.65, 0.60]
 Git: [0.60, 0.55]
 Didactics: [0.30, 0.80]
 Integrated Project: [0.50, 0.90]

Acquired skills :

Session	Skill	Level attained
01	Module architecture	Mastery
02-03	Python OOP (classes, inheritance, encapsulation)	Mastery
04	Advanced algorithms (recursion, sorting)	Advanced
05	Databases and SQL	Mastery
06	Web Development (HTML/CSS/JS)	Advanced
07	Git and collaboration	Mastery
08	Computer science didactics	Awareness
09-10	Integrated project	Synthesis

Objectives achieved :

- ☐ Object-oriented programming in Python
- ☐ SQL queries and database
- ☐ Functional Web application
- ☐ Git repository with history
- ☐ Pedagogical session sheet
- ☐ Presentation and demonstration

7. Next steps (5 min)

Practical internship :

- Observation internship in a school (2 weeks)
- Apply computer science didactics in practice
- Test session sheets with students

Digital portfolio :

- GitHub Pages site showcasing your projects
- Link to the ENS2 project repository
- CV and cover letters

Certifications :

- PIX (digital skills certification)
- C2i (Computer and Internet Certificate)
- TOEFL/IELTS (if planned)

8. Conclusion (5 min)

Congratulations ! The ENS Computer Science module has given you a solid foundation to teach digital literacy in the 21st century. You leave with :

Competent ENS Teacher
✓ Algorithms and programming
✓ OOP and software architecture
✓ Databases and SQL
✓ Web Development
✓ Git and collaboration
✓ Computer science didactics
✓ Digital ethics and security
✓ Integrated project

Computer science is not just a subject — it is a cross-curricular skill for all teachers.

MCQ Self-Assessment

1. What is the duration of the oral project presentation ?
a) 5 minutes b) 10 minutes ✓ c) 20 minutes
2. What is the weight of OOP code in the evaluation ?
a) 10 % b) 20 % ✓ c) 30 %
3. Which error should be anticipated during the demo ?
a) Adding a grade > 20 → error message ✓ b) The server crashing c) The database deleting itself
4. What is the purpose of the pedagogical session sheet in the project ?
a) Describe the code architecture b) Show how to teach the project to middle/high school students ✓ c) List known bugs
5. After the ENS2 module, which digital certification is recommended ?
a) TOEFL b) PIX ✓ c) CCNA

Conclusion

Congratulations ! You have completed the ENS Year 2 Computer Science module. You leave with solid skills in object-oriented programming, algorithms, databases, Web development, Git versioning, and computer science didactics. These tools will accompany you throughout your teaching career and in preparing your future students for digital challenges.

Next steps : Practical internship in a school, building your teacher portfolio, and PIX/C2i certifications.

Pedagogical Note ENS

Teaching transposition : This defense session is itself a model for teachers: it shows how to structure a project evaluation (transparent criteria, live demo, technical questions, pedagogical sheet). This format can be adapted for middle school students (5-minute

Scratch project defense) or high school students (15-20 minute NSI defense). Criteria-based grid assessment is a recommended practice in the official curriculum.

Trainer advice : The defense is a key moment: it evaluates not only the code but also the ability to communicate and argue. Encourage students to prepare a script and rehearse. Provide a projector for the demo and test it beforehand. The technical questions (section 4) should be adapted to each student's level. A weaker student can be asked basic questions (difference class/instance) while an advanced student can be questioned about architectural choices.

Extension : The ENS2 project becomes the centerpiece of the teaching portfolio. Students are encouraged to :

- Present the project during interviews for teaching positions
- Improve it during the practical internship with student feedback
- Publish it on GitHub Pages as a demonstration of their skills
- Extend it with features inspired by their internship experience